Grantham Taylor
granthamtaylor@icloud.com
github.com/json2vec/json2vec

1. Background

Nearly four years ago, while I was at Capital One, I had lunch with my organization's VP. He asked me a simple question that shaped the next several years of my work:

> *Is there a better way to model complex business data?*

The organization had just spent eighteen months building the data engineering capabilities required to produce a *single* high-value feature for a tabular gradient-boosted fraud model (discussed in more detail in Section 5.1). That same year, the team had to scale back a different feature for another model because that static, tabular feature would have required approximately $1 million each year in compute alone.

This is a common but under-discussed constraint in applied machine learning: some problems, at scale, are limited less by the learning algorithm than by the modeling paradigm around it.

Business data rarely starts as a clean table. It is usually nested, historical, heterogeneous, and relational: customers have accounts, accounts have transactions, customers have login sessions, sessions have clickstream events, and every level may contain useful signal in the form of data fields. Traditional modeling workflows force practitioners to flatten that structure into handcrafted tabular features. The result is expensive, time-consuming, error-prone, and difficult to keep consistent between training and real-time serving.

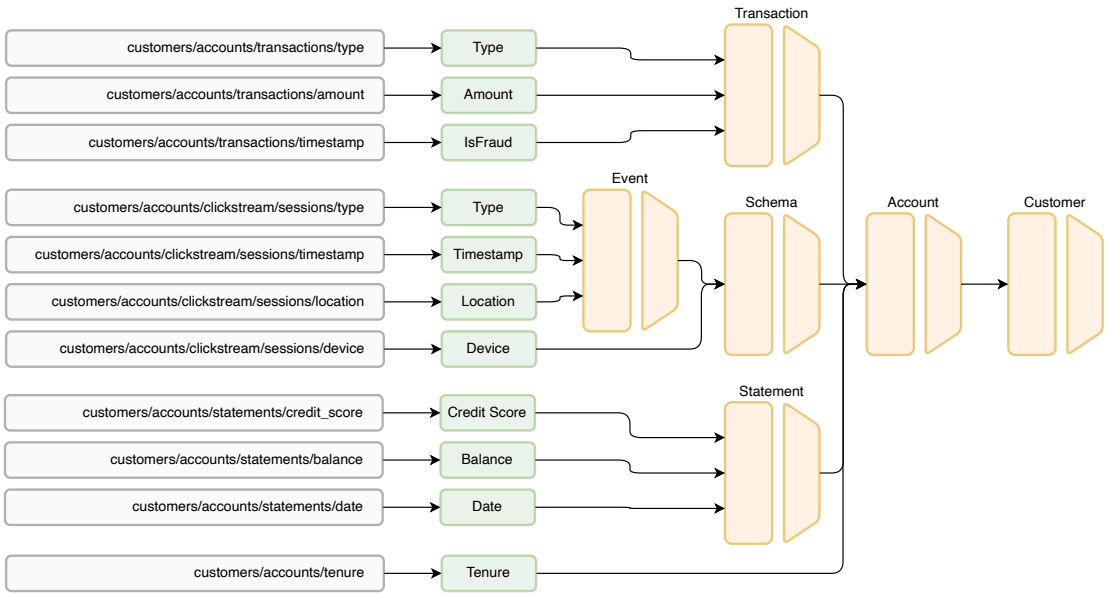

Figure 1: Example of a nested business-data structure.

What my VP was really asking was this: Is there a way to model complex business data without having to resort to tabular reductions of nested relationships?

Teams have pursued this problem through sequence and representation modeling. When I was at Capital One, I attempted several approaches in this space for fraud use cases. A promising design direction is to use hierarchical encoder blocks for collections of nested contexts; this paper presents

that direction as an architectural motivation, not evidence that the industry has converged on one implementation.

However, these implementations are often rigid, proprietary, or inaccessible to developers. In practice, they tend to lack six core components:

1. **Dynamic model architecture**: Model architecture is usually hard-coded or limited to a strict subset of possible topographies, which limits reuse across domains. See Dynamic Model Architecture Instantiation 2.1
2. **Hierarchical context encoding**: Most systems cannot naturally represent multiple nested contexts, such as monthly statements, transactions, login sessions, and clickstream events. See Hierarchical Context Encoding 2.2.
3. **Transfer learning**: Business foundation models are hard to reuse if their schemas cannot evolve as teams add or remove features and targets. See Transfer Learning with Schema Evolution 2.3.
4. **Typed datatype support**: Real business data needs specialized support for categories, numbers, text, entities, embeddings, and dates. The current external custom-datatype registry remains experimental and is not artifact-compatible. See Datatype Plugin Architecture 2.4.
5. **Explainability**: Business models often operate on sensitive decisions, so developers need ways to inspect model behavior beyond a single opaque prediction. See Explainability 2.5.
6. **Integrated querying and transformation**: Source data arrives in inconsistent shapes and formats, so developers need flexible querying and transformation without maintaining a separate feature pipeline. See Integrated Querying, Wrangling, and Logging 2.6.

`json2vec` is a modeling framework I have been developing for several years to address all of these gaps. At a high level, `json2vec` is built around one idea: complex business data should be modeled in its natural shape, and model developers should only need to describe a data schema to instantiate a model that can encode it and make predictions from it.

Instead of flattening nested records into handcrafted feature tables or a single, flattened context window with discrete tokens, `json2vec` enables developers to describe the structure of the data directly. The same schema defines what the model sees, what it can predict, how it can be adapted, and where its intermediate representations live.

The model architecture is constructed dynamically from this schema, including all necessary parameters and the control flow for data streaming, pretraining, finetuning, and both real-time and batch inference.

Combining these processes with the six capabilities above produces a generalizable framework that manages the full modeling lifecycle. With this framework, a foundation model can be pretrained on broad business behavior, adapted as the schema changes, finetuned for specific targets, served using the same data contract, and inspected at the same hierarchical levels used to define the problem.

A pretrained business foundation model will almost always be tied to a particular organization, data contract, risk tolerance, and regulatory environment. A shared framework, however, can be reused without requiring organizations to share private customer data or adopt the same internal feature pipeline.

That creates an opportunity for collaboration across industries. Organizations can contribute shared schema patterns, experimental datatype prototypes, model components, evaluation harnesses, synthetic datasets, and benchmark tasks. A stable external datatype SDK remains future work.

The goal is not that every organization uses the same model. The goal is that they can use the same modeling language and benchmark surface. If the architecture, data contracts, and evaluation tools

are open-source, the community can make measurable progress on structured business-data modeling instead of fragmenting into incompatible internal systems.

The rest of this document describes how `json2vec` fulfills these requirements and gives organizations a shared way to instantiate, pretrain, finetune, and deploy structured-data encoders.

2. Requirements

Generalizability breaks down when any of the following capabilities is missing.

Without **dynamic model architecture**, **hierarchical context encoding**, and typed datatype support, model developers cannot express the range of architectures required for complex structured data. Developers need a succinct way to describe a target structure and use supported built-in datatypes. A stable create-your-own-datatype contract requires persistence and compatibility guarantees that the current experimental registry does not yet provide.

However, architecture flexibility alone is not enough. An organization's foundation model is only practical across many use cases if its schema can evolve. Teams need to add and remove fields as their use cases change. An *overloaded* foundation model is pretrained on every available data field; it may be slow, expensive, unwieldy, and inappropriate for sensitive use cases, such as credit decisioning with personal information. An *underloaded* foundation model is trained only on universally relevant fields; it may underperform task-specific models that use handcrafted features for the task at hand. Without use-case-specific information, such as device identifiers or biometrics for fraud, a foundation model may supplement model development but cannot act as a standalone implementation.

Schema mutation provides a middle ground. Organizations can create and maintain foundation models with an appropriate set of general fields, then adapt them into child foundation models or task-specific models that add relevant fields and remove unnecessary ones. Developers can also explore the impact of removing fields at inference time with integrated ablations 2.5.1, one of several **explainability** techniques that `json2vec` prioritizes.

Finally, even a powerful, flexible, and mutable architecture is not enough if batch data processes still dominate the model-development lifecycle. `json2vec` therefore supports configuration-based data querying and registered user-defined functions that transform raw data inside the training and inference paths, reducing the need for separate batch feature pipelines. These techniques are described in more detail in Section 2.6.

2.1. Dynamic Model Architecture Instantiation

As mentioned previously, the schema is the basis of modeling with `json2vec`.

A schema defines the contexts, fields, and datatype-specific settings that determine the model architecture.

> **Note.** For simplicity, I use `yaml` to illustrate schemas. The examples in this document are conceptual schema snippets, not complete runnable configuration files. Full experiment configs include additional project, session, dataset, trainer, and deployment settings. Under the hood, `json2vec` loads schemas as `pydantic` models backed by an `AnyTree` structure, then uses them to initialize the model. Every context is modeled as a sequence, including the root context, which is always a list of one. That is why field queries start with `[*]` even when the sample input looks like a single record.

2.1.1. Transformer Architecture Background

Transformers are useful because they learn relationships among items within a context window. In language models, those items are usually tokens in a sentence. Each token is embedded into a vector,

the transformer lets tokens attend to one another, and the resulting representation is used to reconstruct masked tokens, generate text, classify a sentence, or solve another downstream task.

The core mechanism is self-attention. Each item produces a query, key, and value vector. Attention compares queries to keys to decide which items are relevant, then mixes the corresponding values into a new contextual representation. Repeating this across multiple heads, feed-forward layers, and stacked blocks allows the model to learn different kinds of relationships at the same time.

`json2vec` applies the same general idea to structured business data. Instead of assuming that the only meaningful sequence is a sentence, the schema defines the contexts that should be modeled: transactions in an account, statements in a customer history, login sessions for a user, clickstream events inside a login session, pieces on a chess board, or characters inside a product code.

Each field is first converted into an embedding. A context encoder then allows the child embeddings inside that context to exchange information through attention. Finally, a pooling step compresses the context into one or more vectors that can be passed upward to the parent context.

`json2vec` instantiates a hierarchy of transformer encoders. Leaf fields become vectors, child contexts become summarized vectors, parent contexts consume those summaries, and the root representation is informed by every level below it. The shift is that the model is no longer limited to one flat row or one flat sequence; it can encode nested `json`-like structures directly.

That hierarchy is what makes the architecture different from a standard tabular model. A transaction can be modeled alongside other transactions in the same statement. The statement can then be summarized alongside other statements in the customer history. The model learns at each level before passing information upward, rather than forcing every raw value to compete inside one very wide feature vector.

This is why the schema matters so much. It tells the model which values should attend to each other locally, which contexts should be summarized, how those summaries should flow upward, how inputs should be embedded, and which fields should be predicted. In other words, the schema is not just metadata around the model; it is the model's blueprint.

2.1.2. Hello World Example

With `json2vec`, one can easily define a basic tabular model with a schema like so:

```yaml
name: record
n_layers: 4
n_heads: 4
fields:
  - name: x2
    type: category
    size: 1000
    query: "[*].x2"

  - name: x1
    type: category
    size: 1000
    query: "[*].x1"
```

This model has a single context with two tabular inputs (`x1` and `x2`). Both inputs are categorical and may learn up to 1000 unique values.

Sample input:

```
{ "x1": "my_value", "x2": "my_other_value" }
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

These two categorical inputs are processed as follows:
1. Tokenized (using a novel online vocabulary mechanism described in Section 2.4.1)
2. Embedded into vectors of width `d_model` (defined elsewhere)
3. Passed to a transformer encoder block (`record`) with 4 layers and 4 heads
4. Pooled together with a cross-attention block
5. Reconstructed from the embedding during pretraining and/or finetuning

During pretraining, the model will randomly mask values according to a masking rate hyperparameter. It will then attempt to impute the masked values from the remaining available information.

This is similar in nature to masked language modeling (MLM). While each training observation has only one value for `x1` and one value for `x2`, each training batch contains many such observations. By masking different values across a batch, the model learns to reconstruct `x1` from `x2`, `x2` from `x1`, or either field from the learned prior when no paired value is available. The result is a generalizable representation of the data structure, and developers can later `prune` `x1` or `x2` and finetune the model to specialize in either task.

2.1.3. Basic BERT-like Model

Similarly, one can build a model like BERT by defining a nested context:

```yaml
name: observation
n_layers: 1
n_heads: 4
fields:

  - name: context
    n_layers: 8
    n_heads: 4
    context_size: 768
    fields:

      - name: tokens
        description: each unique wordpiece token
        type: category
        size: 20000
        query: "[*].tokens[*]"
```

Sample input:

```
{ "tokens": ["hello", "world"] }
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

This model architecture is functionally similar to BERT. Encoding text like this is more of a thought exercise in nested contexts because, in practice, developers can use the dedicated `text` datatype in `json2vec`, which uses pretrained (BERT) models from Hugging Face.

During pretraining, the model randomly masks tokens and imputes the masked wordpieces from the surrounding context.

Additionally, the input requires a list of wordpiece values. A developer implementing a BERT-like model with `json2vec` could use a pre-built wordpiece auto-tokenizer inside custom transformation functions, discussed further in Section 2.6.

The schema can also include fields beyond wordpiece tokens:

```yaml
name: observation
n_layers: 4
n_heads: 4
fields:

  - name: context
    n_layers: 8
    n_heads: 4
    context_size: 768
    fields:

      - name: tokens
        description: each unique wordpiece token
        type: category
        size: 20000
        query: "[*].tokens[*]"

      - name: part_of_speech
        description: part of speech for each word (verb, noun, adjective, etc.)
        type: category
        size: 100
        query: "[*].part_of_speech[*]"

  - name: sentiment
    description: sentiment of message (positive, neutral, negative)
    type: category
    size: 3
    query: "[*].sentiment"
```

Sample input:

```
{
  "tokens": ["this", "works"],
  "part_of_speech": ["pronoun", "verb"],
  "sentiment": "positive"
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

This illustrates an important point: `json2vec` can create a family of models, including BERT-like models.

However, the architectures instantiated from the pipeline are flexible. Inputs and outputs are defined in the same schema. Developers can mark any field as an output target that the other fields must reconstruct.

For example, developers can pretrain a model with stochastic reconstruction, then make `sentiment` and `part_of_speech` supervised targets while keeping `tokens` visible:

```
model.update(jv.where("name") == "sentiment", target=True)
model.update(jv.where("name") == "part_of_speech", target=True)
```

`target=True` is the public shorthand for `p_prune=1.0`: the selected leaf is always hidden from encoder input while its original value remains the training target. After finetuning, input may omit these target fields and the model will decode them from the visible wordpiece tokens.

The same underlying code handles pretraining and finetuning. This is discussed in more detail in Section 3.1.

In short: a supervised finetuning configuration is a special case in which a subset of fields use `target=True`; other fields may remain visible or retain intentional stochastic reconstruction objectives.

2.1.4. Basic Chess Encoding

In the same way `json2vec` can build tabular models, or a superset of `BERT` models with arbitrary outputs, it can also model chess positions.

It can do this by representing each board as a fixed-size context and pairing it with the score of the position at that point in time. By training on observed games, the model can learn to estimate an evaluation from the current board snapshot rather than replaying the full history of the game.

```yaml
name: observation
n_layers: 4
n_heads: 4
fields:

  - name: board
    n_layers: 8
    n_heads: 4
    context_size: 64
    description: the board is a flattened 8x8 grid.
    fields:

      - name: piece_type
        description: >
          each unique piece type (pawn, bishop, knight, rook, queen, king)
          empty squares are marked by `None`
        type: category
        size: 6
        query: "[*].board[*].piece_type"

      - name: piece_color
        description: >
          player colors (black & white)
          empty squares are marked by `None`
        type: category
        size: 2
        query: "[*].board[*].piece_color"

        # consider adding castling rights as additional context

  - name: player_to_move
    type: category
    size: 2
    query: "[*].player_to_move"
```

```yaml
  - name: centipawn_score
    description: centipawn score of position
    type: number
    query: "[*].centipawn_score"
```

Sample input:

```
{
  "board": [
    { "piece_type": "rook", "piece_color": "white" },
    { "piece_type": "knight", "piece_color": "white" },
    { "piece_type": null, "piece_color": null },
    { "piece_type": "king", "piece_color": "black" }
  ],
  "player_to_move": "white",
  "centipawn_score": 0.32
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

Pretraining, in this case, means randomly masking individual piece attributes, such as color and type, and training the model to reconstruct the missing components of the game snapshot from the available information.

Upon finetuning, the model can take the available board information and predict an evaluation directly from the fixed-size position representation.

The same flexibility can support related targets. For example, the model could take the state of the board and predict `player_to_move` instead. This is a slightly different modeling problem that can reuse transfer learning alongside the original task.

2.2. Hierarchical Context Encoding

Many architectures already support tabular inputs or a single sequence-like context.

`json2vec` supports multiple contexts, each of which may have its own child contexts. I refer to this as hierarchical context encoding. The implementation details of how information moves through this tree are described in Section 3.2.

Hierarchical context encoding is not just a technical detail. It is useful in practical settings. For example:

- Clickstream events within login sessions
- Purchased items within purchase orders

Moreover, it can enable vocabulary sharing that would otherwise be awkward or impossible, such as complex string deconstruction 2.2.1 and field stacking 2.2.2. It also helps address a sequence-model vulnerability I refer to as the *flushing problem* 2.2.4.

2.2.1. Complex String Deconstruction

Strings are just a context of characters. Textual data assumes that strings are better represented as wordpieces, but in some business problems they simply are not. For example, some product IDs may contain semantic information that is encoded character-by-character. Creating an entire embedding for each unique combination of characters doesn't capture the semantic information available within the values.

For example, I have found value in breaking such strings down into a list of characters while working with "Fare Basis Codes" in the context of aviation. These are 2- to 16-character strings that roughly describe an itinerary's contract. There are over a hundred thousand possible combinations, and there is a lot of available information within the individual characters.

The following is a naive implementation of encoding fare basis codes.

```yaml
name: itinerary
n_layers: 4
n_heads: 4
fields:

  - name: fare_basis_code
    type: category
    size: 30000
    query: "[*].fare_basis_code"
```

Sample input:

```
{ "fare_basis_code": "Y26NR" }
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

However, it is much more efficient to represent fare basis codes with the following.

```yaml
name: itinerary
n_layers: 4
n_heads: 4
fields:

  - name: fare_basis_code
    n_layers: 4
    n_heads: 4
    context_size: 16
    fields:

    - name: characters
      type: category
      size: 100
      query: "[*].fare_basis_code_chars[*]"
```

Sample input:

```
{
  "fare_basis_code_chars": ["Y", "2", "6", "N", "R"]
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

In practice, a preprocessor can derive `fare_basis_code_chars` from the original string before encoding, so source systems do not need to store the data in this exact shape. This can be done with streaming transformation functions, further discussed in Section 2.6.2.

Naturally, developers can also represent multiple fare basis codes with additional context blocks. The broader point is that nested contexts are far more common than one might expect.

2.2.2. Stacking Field Embeddings

In some cases, developers can encourage the model architecture to share parameters among attributes using an emergent pattern I refer to as “field stacking”.

Consider the following example of a travel itinerary:

```yaml
name: itinerary
n_layers: 4
n_heads: 4
fields:

  - name: origin
    type: category
    size: ...
    query: "[*].origin"

  - name: destination
    type: category
    size: ...
    query: "[*].destination"
```

Sample input:

```
{
  "origin": "IAD",
  "destination": "SFO"
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

This schema is simple and easy to read, but it is harder for the model to understand because it needs to learn distinct embeddings for both `itinerary/origin` and `itinerary/destination`. Developers can simplify this by stacking the origin and destination into a new context, which lets both positions share embeddings:

```yaml
name: itinerary
n_layers: 4
n_heads: 4
fields:

  - name: locations
    n_layers: 1
    context_size: 2
    fields:

      - name: location
        type: category
        size: ...
        query: "[*].[origin, destination]"
```

Sample input:

```
{
  "origin": "IAD",
  "destination": "SFO"
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

Now, `itinerary/locations/location` will share parameters. This requires no change to the source data because the `jmespath` query reshapes the values at encode time. Querying is discussed further in Section 2.6.1.

`jmespath` querying enables succinct extraction of data from complex `json`-like data structures without modifying the data on the fly. Every field requires a `query` for this reason.

Broadly speaking, support for `jmespath` is meant to enable modeling from many source shapes: a dictionary of lists, a list of dictionaries, a dictionary of dictionaries of lists, or whatever else appears in the source system.

2.2.3. Fraud Detection

The examples so far have been fairly small. Hierarchical context encoding becomes more valuable as the data becomes more complex. Developers can define rich schemas for deeply nested data structures.

The following example uses multiple contexts, including one nested inside another.

```yaml
name: customer
n_layers: 4
fields:

  - name: transaction
    n_layers: 6
    description: up to 512 most recent trailing transactions
    context_size: 512
    fields:

      - name: type
        description: transaction type (card swipe, ACH, wire, etc.)
        type: category
        size: 20
        query: "[*].transactions[*].type"

      - name: amount
        description: transaction amount
        type: number
        query: "[*].transactions[*].amount"

      - name: timestamp
        type: dateparts
        # dateparts extract parts from dates / timestamps
        dateparts:
        - day_of_week
        - day_of_month
        query: "[*].transactions[*].timestamp"

  - name: statement
    n_layers: 4
    description: up to five years of trailing monthly statements
    context_size: 60
    fields:
```

```
      - name: balance
        type: number
        query: "[*].statements[*].balance"

      - name: fees_accrued
        type: number
        query: "[*].statements[*].fees_accrued"

      - name: total_spent
        type: number
        query: "[*].statements[*].total_spent"

  - name: login_sessions
    n_layers: 1
    description: up to 24 trailing login sessions
    context_size: 24
    fields:

      - name: device
        description: device used for login session - helpful for modeling fraud
        type: entity
        query: "[*].login_sessions[*].device"

      - name: region
        type: category
        description: region / state of device used for login session
        size: 20
        query: "[*].login_sessions[*].region"

      - name: clickstream_events
        n_layers: 2
        description: set of events happening within each login session
        context_size: 128
        fields:

          - name: type
            description: clickstream event type
            type: category
            size: 20
            query: "[*].login_sessions[*].clickstream_events[*].type"

          - name: timestamp
            type: dateparts
            # dateparts extract parts from dates / timestamps
            dateparts:
            - hour_of_day
            - minute_of_hour
            query: "[*].login_sessions[*].clickstream_events[*].timestamp"
```

Sample input:

```
{
  "transactions": [
    {
      "type": "card_swipe",
      "amount": 42.13,
      "timestamp": "2026-04-30T14:05:00"
```

```
    }
  ],
  "statements": [
    {
      "balance": 1200.52,
      "fees_accrued": 8.25,
      "total_spent": 530.10
    }
  ],
  "login_sessions": [
    {
      "device": "device_hash_123",
      "region": "VA",
      "clickstream_events": [
        {
          "type": "forgot_password",
          "timestamp": "2026-04-30T13:57:00"
        },
        {
          "type": "change_email",
          "timestamp": "2026-04-30T13:59:00"
        }
      ]
    }
  ]
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

This schema may be pretrained on slices of a customer's event history: time-windowed snapshots of observed behavior. This can be done at scale by streaming customer data, sampling a time window, and filtering the data down to that window. One customer may yield multiple observations, but it is typically prudent to prevent leakage by stratifying training, validation, and testing data by a unique customer identifier.

After pretraining the model on customer behavior, developers can finetune multiple fraud models with different tagging strategies at different levels. For example, they may create the field `customer/transaction/is_account_takeover_fraud` at the transaction level and then `prune` it so the model focuses on imputing whether each transaction is indicative of account takeover fraud. Alternatively, they may create the field `customer/is_first_party_fraud` to predict first-party fraud at the customer level.

Keep in mind that nested contexts require significant GPU resources. Shaping the transformer encoder blocks, including input pooling, number of heads, and number of layers, becomes critical for keeping the model performant and avoiding out-of-memory errors.

2.2.4. The Flushing Problem

While working on fraud models at Capital One, I came across an attack pattern that I now think of as *the flushing problem*.

Many production models use a fixed-size trailing window: the last 400 transactions, the last 20 login events, the last 50 device events, and so on. In adversarial settings, a bad actor can sometimes exploit that design by creating low-value activity that pushes more important events out of the model's visible context.

For example, an account takeover attempt may include meaningful signals such as `forgot password`, `change email`, new-device login, or unusual transfer setup. If the model only sees a flat trailing window, an attacker may be able to dilute or *flush* that context by repeatedly logging in and out, generating harmless clickstream events, or sending many small transfers.

Hierarchical context encoding is one possible mitigation to test. Instead of forcing all behavior into one flat sequence, a schema can preserve separate windows for transactions, login sessions, and clickstream events within each session. That design may keep a suspicious password-reset and email-change flow local to its session even when later activity creates noise elsewhere.

Because there are multiple context windows, flushing behavior can be separated by frequency, relevance, and sensitivity, allowing the most important events to live in different contexts.

This is a design hypothesis, not a demonstrated security guarantee. Evaluate it with explicit flush-style perturbations, branch truncation measurements, and a flat-window baseline before claiming that it reduces attack paths.

Custom preprocessing functions defined in Section 2.6.2 provide another mitigation. Developers can programmatically filter out irrelevant events during training and inference before they enter the model context.

2.3. Transfer Learning with Schema Evolution

The schema is the basis of modeling with `json2vec`. The schema is meant to be flexible and adaptable to accommodate changes to upstream data.

`Model.load(...)` reconstructs the schema saved in the artifact; it does not accept a replacement schema. After loading, the public mutation methods can evolve that model. Compatible parameters and field-owned state are retained when their address, type, and tensor shape still match, while new or shape-incompatible modules are initialized. The evolved model must be trained, evaluated, and saved as a new artifact.

Fields may be added and removed because each parent context has a flexible context width.

Enterprise organizations may build foundation models with the most generalizable fields, then share the checkpoints with individual data science teams. These teams can add use-case-specific fields, continue pretraining, and eventually finetune to their targets.

Once the data changes, whether because of a new schema or new customer behavior, the organization can refit the foundation model and share an updated checkpoint for downstream teams to adapt and refit for their use cases.

Without schema evolution, the organization would need to resort to one or both of the following unpleasant alternatives:

1. Integrate only a subset of fields to maximize generalizability
2. Manage and periodically train multiple foundation models independently (wasting compute)

By using transfer learning with schema evolution, teams can adapt foundation models with new fields for their individual use cases.

2.4. Datatype Plugin Architecture

Schemas define the shape of the model, but datatype plugins define how each data field behaves. The built-in datatypes use this architecture internally. Although registry and base objects are importable, dynamically registered external request types do not currently round-trip through saved schema validation; treat custom tensorfields as an experimental, same-process extension surface rather than a stable artifact-compatible plugin SDK.

A field's `type` is not only a validation hint. It selects a small bundle of components that know how to:

- Validate datatype-specific schema parameters
- Convert raw `json`-like values into tensors
- Mask and prune values during training
- Embed the field into the shared `d_model` representation
- Decode model context back into datatype-specific predictions
- Compute losses for masked or pruned targets
- Serialize predictions for inference and evaluation

This is the key abstraction that allows `json2vec` to model categories, numbers, timestamps, text, embeddings, and entities with the same high-level training loop. The context encoder does not need to know whether a field started as a string category, a floating-point value, a timestamp, or a pretrained text embedding. By the time the value enters the architecture, the plugin has converted it into a parcel of vectors. By the time the model produces a prediction, the plugin owns how to score and write that prediction.

Conceptually, a datatype plugin for `foo` looks like this:

```
foo: Plugin = Plugin(name="foo")

@foo.register
class Request(RequestBase):
    type: Literal["foo"]
    # datatype-specific schema

@foo.register
class TensorField(TensorFieldBase):
    # a complex, multi-attribute tensorclass
    # used to represent encoded content, state, trainable mask, and cached targets

@foo.register
class Embedder(EmbedderBase):
    # convert tensorclass into embedding parcel

@foo.register
class Decoder(DecoderBase):
    # context parcels -> datatype-specific prediction tensors

@foo.register
def loss(module, prediction, batch, strata):
    # datatype-specific supervised/self-supervised loss function and logging logic

@foo.register
def write(module, prediction):
    # datatype-specific inference output
    # optional when the datatype has no public decoded payload
```

The important point is that the architecture receives a uniform interface while the datatype plugin remains free to be specialized.

A `number` plugin can use continuous regression losses, a `category` plugin can use cross-entropy over a bounded vocabulary, a `dateparts` plugin can decompose timestamps into calendar components, a `text` plugin can call a pretrained Hugging Face encoder, and a `vector` plugin can learn against distances from dense embeddings.

This design keeps built-in datatypes specialized without forcing every value into the same crude representation. A future stable extension SDK can expose the same component boundary once registration, persistence, compatibility, and distributed loading have explicit guarantees.

Developers may, in the future, implement `image`, `video`, or `audio` datatypes, but media fields require more deliberate file, object-store, and batching semantics than the current core examples cover.

2.4.1. Online Categorical Vocabulary

Categorical data creates a practical problem: most business datasets have string labels whose vocabulary is either unknown ahead of time or too inconvenient to fully materialize before training.

The `category` datatype handles this with an online vocabulary tokenizer. During training, observed labels are assigned integer ids until `size` is reached. The learned vocabulary becomes part of the model state, so validation, testing, finetuning, and inference can reuse the same mapping.

The model will never learn vocabulary observed outside of training, which could lead to unexpected behavior.

When a category appears outside the learned vocabulary, `json2vec` does not treat the field as missing. Its state remains `valued`, while tensorization uses an internal unavailable sentinel. The sentinel is not an embedding-table row or output class: unavailable categorical content contributes a zero content vector, leaving the separate `valued` state embedding to record that a source value was present. A transaction with a new merchant category is therefore different from a transaction with no merchant category at all.

Training can deliberately replace a small fraction of known category content with the same unavailable condition through `p_unavailable`. If such content becomes a reconstruction target, it uses a uniform objective over the real classes and is excluded from categorical content accuracy. The decoder still scores exactly `size` real labels; unavailable is never emitted as a predicted label.

For example:

```yaml
- name: merchant_category
  type: category
  size: 5000
  p_unavailable: 0.01
  topk: [5, 20]
  query: "[*].merchant_category"
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

This field learns up to 5000 real labels and can optionally report top-k alternatives from the populated training vocabulary during prediction. Validation, test, and prediction inputs never expand that vocabulary. See the current Category reference for the complete runtime, metric, and output contract.

2.4.2. Unified Enumerable State Management

Every datatype needs to represent more than content. It also needs to represent whether the content exists, whether it was padded, or whether it was deliberately hidden.

`json2vec` handles this with a shared state vocabulary:

- `valued`: a source value exists, even if vocabulary-backed content is unavailable
- `null`: the source value is explicitly absent (`None`)
- `padded`: the value was introduced only to fill a fixed context shape

- `masked`: the input value is hidden by masking, pruning, or a supervised target
- `other`: a reserved state for datatype-specific extensions

Each `TensorField` therefore carries four pieces of information:
- `content`: the datatype-specific tensor representation
- `state`: the enumerable state token for each position
- `trainable`: which positions should contribute to loss
- `targets`: cached original values used when hidden positions are decoded

This is what makes the training and finetuning path the same path. Pretraining masks sampled positions; pruning and `target=True` hide whole leaf instances. The current runtime applies configured masking and pruning in train, validation, and test; prediction bypasses those stochastic policies while supervised targets are still supplied as hidden inputs. All hiding operations set the selected *input* state to `masked`, cache the original target, and mark trainable positions. Pruning is an operation, not a sixth state token. During prediction, a separate public `inferred` mask says which decoded positions were masked in the request.

Because this state system is shared, new datatypes get masking, pruning, padding, and missing-value behavior without inventing their own control flow. The datatype still decides what its `content` tensor means, which selected states contribute to content loss, and how to write decoded predictions. See the current Data Types reference for the canonical state and prediction-envelope contract.

2.4.3. Built-In Entity Encoding

Entity fields are for identifiers where the exact value matters only in relation to other values in the same observation. Examples include devices inside login sessions, accounts inside a transfer graph, merchants inside a transaction window, or repeated users inside a collaboration event (complex, many-to-many relationships such as multiple accounts per customer, or multiple customers per account).

These values are usually high-cardinality and unstable. Treating them as ordinary categories can waste vocabulary capacity, while treating them as raw strings can make generalization brittle.

The `entity` datatype instead locally re-indexes hashable scalar values within each encoded observation and Entity leaf. If the same `device_id` appears in three login sessions in the same observation and leaf, those positions receive the same local entity index. A different device receives a different local entity index. Indexing restarts at zero for every observation, follows first-seen order, and does not align across separate Entity leaves, observations, batches, or checkpoints.

This gives the model a way to learn sameness, repetition, and co-occurrence patterns without maintaining an enormous global entity vocabulary. In other words, an `entity` defines an ephemeral, machine-readable temporary identifier. It allows the model to compare repeated values within one leaf without learning anything specific about the object globally. Cross-collection identity requires restructuring or stacking both roles into that same leaf; two sibling Entity fields do not share a code space.

For example:

```
- name: login_sessions
  n_layers: 1
  context_size: 24
  fields:

    - name: device
```

```
      type: entity
      query: "[*].login_sessions[*].device"

    - name: region
      type: category
      size: 100
      query: "[*].login_sessions[*].region"
```

Sample input:

```
{
  "login_sessions": [
    { "device": "device_hash_123", "region": "VA" },
    { "device": "device_hash_123", "region": "VA" },
    { "device": "device_hash_987", "region": "CA" }
  ]
}
```

> **Note.** Conceptual query/input pair only. Shape the raw input and `jmespath` query around your own data; the query is the bridge from your record shape into the schema. See Querying with JMESPath 2.6.1.

In this schema, `device` helps the model reason about whether the same device recurs across sessions, while `region` remains a conventional bounded category.

This distinction is useful in fraud and abuse problems, where exact identifiers often churn but repeated relationships are highly predictive. For account takeover, one of the strongest signals is whether the current login session comes from a familiar device. If the device also appears in login sessions from days, weeks, or months earlier, the session is more likely to be legitimate. If the device has never appeared before, the risk of account takeover may be much higher.

A similar pattern can help financial institutions use geographic context more carefully. Geography can be useful for legitimate reasons: fraud detection, branch access, merchant location, travel patterns, device risk, regional economic shocks, disaster response, and operational monitoring. But in lending, insurance, and other sensitive financial decisions, raw geography can also become a proxy for protected characteristics or historically discriminatory boundaries.

The goal is not to let the model quietly learn a redlining map. The goal is to represent geographic information at the right level of abstraction, for the right task, with enough structure to audit how it is being used.

For example, a fraud model may need to know that a cardholder usually transacts in northern Virginia and suddenly appears in another country. A credit model, however, should not learn that a neighborhood alone is a reason to deny credit. Structured geographic representations can help models reason about differences between card-swipe locations without exposing exact locations as direct decision features.

2.5. Explainability is built-in

2.5.1. Via Pruning

`json2vec` treats pruning as a first-class modeling operation, not as an external ablation script.

This matters because the same mechanism used for supervised learning can also be used for explanation. When a field is pruned, its observed value is removed from the model input and cached as a target. The model must reconstruct that value from the remaining context. This makes a pruned field a natural question:

> *Given everything else in this observation, what does the model believe this hidden field should be?*

For a fraud model, this can be used in several ways:

- Prune `customer/transaction/is_fraud` to train or evaluate the fraud target.
- Prune `customer/transaction/amount` to understand whether the surrounding context implies an unusual transaction amount.
- Prune an entire class of fields across experiments and measure degradation in target quality.

The last case is especially useful. Because fields and contexts have stable addresses, a developer can run controlled experiments where a branch is removed and all other training settings remain fixed. If pruning `customer/login_sessions/clickstream_events` significantly harms account-takeover detection, that is a direct signal that clickstream behavior is contributing useful information. If pruning it has no effect, the branch may be low-signal or redundant.

This is not meant to claim causal explanation. It is a practical model-behavior diagnostic: remove information, hold the rest of the pipeline constant, and measure how reconstruction, prediction, and embeddings change.

2.5.2. Via Embedding Trees

Every context in the schema produces an intermediate representation. The model can emit more than a single root vector; it can emit embeddings at selected addresses in the tree.

This enables multi-resolution inspection. Two customers may look similar at the root level but diverge sharply inside `customer/login_sessions`. Two transactions may look different at the transaction level but still live inside customers whose monthly statement histories are similar. By requesting embeddings from multiple addresses, developers can compare observations at the level where the difference actually occurs.

For example:

```
embed:
  - customer
  - customer/transaction
  - customer/login_sessions
  - customer/login_sessions/clickstream_events
```

The resulting embeddings form a tree that mirrors the schema. This gives downstream analysis a simple path:

1. Compare root embeddings to find globally similar observations.
2. Compare child context embeddings to localize which branch explains similarity or distance.
3. Compare leaf or lower-level context embeddings to inspect the concrete behavioral pattern.

This is particularly useful for nested business data because the relevant signal is often not located at one level. For example, a model may identify two customers as similar because their login-session trees are similar, not because their transaction amounts are similar. The embedding tree makes that distinction observable.

The model also exposes a Rich representation that follows the same tree, so diagnostics can stay aligned with the schema that owns the representation.

> **Note.** This capability has yet to be tested. Comparing embedding distances across unrelated attribute blocks makes some fairly bold assumptions. I have not yet had the time to explore and validate these assumptions.

2.5.3. Via "What Ifs"

Because `json2vec` works directly from raw, structured observations, counterfactual analysis can be expressed in human terms. Instead of asking which derived feature changed, a practitioner can ask:

- What if this transaction amount had been $500 instead of $50?
- What if the customer had not changed their email before the transfer?
- What if this login session came from a known device?
- What if the last ten tiny transfers were removed?
- What if the customer had one more month of normal statement history?

The workflow is straightforward: copy the raw observation, edit the part of the record that represents the scenario, run the same schema and model again, and compare the prediction or embedding output.

That is much harder in a traditional tabular feature pipeline. If the upstream data is hierarchical, one human-level change can affect many downstream features at once: counts, sums, rolling averages, recency features, velocity features, distinct-device counts, session aggregates, merchant summaries, and dozens of other hand-authored transformations. To simulate a simple question like "what if this login event did not happen?", the practitioner has to know every feature that would have changed as a consequence.

In `json2vec`, the source-of-truth object remains the object being modeled. A login event can be removed from `login_sessions`; a clickstream event can be inserted into `clickstream_events`; a transaction can be edited in `transactions`; a statement can be added to `statements`. The model pipeline then recomputes the representation from the changed observation.

This does not make the result causal by itself. It is still a model-behavior diagnostic. But it makes counterfactual probing far more ergonomic because the question can be stated in the same language as the business event.

For example, an investigator reviewing an account-takeover alert can create variants of the same customer snapshot:

1. Original observation
2. Observation without the `forgot_password` event
3. Observation without the `change_email` event
4. Observation with the suspicious transfer amount reduced
5. Observation with the login session moved back to a known device

If the predicted risk or relevant embeddings change sharply across these variants, the investigator has a concrete path for understanding what the model is reacting to. The explanation is not "feature 182 increased"; it is "the model is sensitive to the password reset and email change immediately before the transfer."

2.6. Integrated Querying, Wrangling, and Logging

The data path is designed so that raw observations, schema-defined extraction, optional wrangling, tensorization, training, inference, and output writing all share one execution path.

This is important operationally. In many production ML systems, training data is prepared by one feature pipeline and real-time inference is prepared by a different service. That separation creates training-serving skew. `json2vec` avoids this by putting extraction and transformation directly into the model pipeline.

2.6.1. Querying with JMESPath

Every leaf field has a `jmespath` query. The query defines how values are pulled from the incoming `json`-like observation before the datatype plugin converts them into tensors.

> **Note.** The queries and sample inputs in this section are intentionally simple. They are not a required data format. `jmespath` is meant to let the schema adapt to the structure you already have: maps of arrays, arrays of maps, deeply nested objects, flattened records, or source-specific payloads. The field query is the bridge between your raw record shape and the model's schema.

For simple fields, the query is usually direct:

```yaml
- name: amount
  type: number
  query: "[*].amount"
```

Sample input:

```json
{ "amount": 42.13 }
```

For nested contexts, queries can reshape values without rewriting the source object:

```yaml
- name: location
  type: category
  size: 50000
  query: "[*].[origin, destination]"
```

Sample input:

```json
{
  "origin": "IAD",
  "destination": "SFO"
}
```

This query can turn two sibling fields into a shared two-position context, allowing `origin` and `destination` to share one vocabulary and one embedding table.

More generally, `jmespath` makes the schema responsible for selecting data while preserving the raw record format. This is useful when source systems produce dictionaries of lists, lists of dictionaries, deeply nested event payloads, or records whose shape is awkward but stable.

The implementation validates queries when schemas are loaded and compiles them for reuse during encoding. It also performs periodic spot checks to catch queries that consistently return empty results, which is one of the easiest ways to silently train a bad model.

2.6.2. Wrangling with Preprocessors

Some data transformations are too domain-specific for a declarative query. Examples include parsing vendor-specific payloads, sampling time windows from a customer history, deriving auxiliary labels, normalizing inconsistent field names, or splitting one raw record into multiple training observations.

For this, `json2vec` supports optional dataset preprocessors. A preprocessor runs before tensorization and receives the raw observation plus explicit named runtime values such as `strata`, `schema`, and `encoding_context`. A preprocessor returns `jv.Observation | None`, or yields zero or more `jv.Observation | None` values from one input.

When no preprocessor is configured, observations pass through unchanged. A custom preprocessor can sit between a messy source system and a clean modeling schema:

```
import json2vec as jv
```

```
@jv.preprocess
def customer_windows(customer, *, window_days: int):
    for window in sample_windows(customer, days=window_days):
        yield jv.Observation({
            "transactions": window["transactions"],
            "statements": window["statements"],
            "login_sessions": window["login_sessions"],
        })


preprocessor = customer_windows.partial(window_days=30)
```

The schema still owns the model-facing contract. The preprocessor only prepares observations into a shape the schema can query. This separation keeps domain wrangling explicit without forcing developers to materialize a separate feature table.

The same preprocessor path is used during training, batch prediction, and real-time serving. That is the key design point: once a preprocessor and schema are paired, the model sees the same transformation logic in every environment.

2.6.3. Logging and Prediction Outputs

Logging is integrated at three levels.

First, the model logs field-level metrics through the same datatype plugins that compute losses. Categorical fields can log accuracy, numerical fields can log error metrics, and every metric is grouped by address and stage. This makes it possible to identify where the model is struggling: not only that validation loss increased, but that `customer/transaction/amount` or `customer/login_sessions/device` became unstable.

Second, the training pipeline logs lifecycle and throughput information. Throughput is tracked in observations per second, which is useful when tuning batch size, dataloader workers, sharding strategy, or remote execution resources.

Third, prediction output is written in an analysis-friendly format. Batch prediction writes parquet records containing:

- the processed, model-facing observation metadata
- supervised predictions
- optional embeddings

This makes offline evaluation straightforward. A developer can train or finetune a model, run prediction over a validation or production sample, and inspect the processed observations alongside the model's reconstructed targets and intermediate embeddings. Raw source fields appear only when the preprocessor retains them in its emitted observation.

The framework can also attach standard experiment trackers when configured, including local CSV or TensorBoard logging and remote systems such as Weights & Biases, Neptune, Comet, or MLflow.

3. Implementation Details

3.1. Unified Self-Supervised and Supervised Learning Tasks

Unifying self-supervised learning with supervised learning simplifies control flow, loss functions, and logging. Because another requirement of this project is the ability to manage an extensible library of datatypes (categories, numbers, text, dateparts, embeddings, entities, etc.), reusing components is critical.

The idea is simple: the same datatype-specific losses are used for self-supervised learning and supervised learning.

During pretraining, all masked values are imputed regardless of their dimensionality. During supervised learning, all targeted values are predicted regardless of their dimensionality. The difference is that masking happens value-by-value according to the masking rate. Targeting removes a field from the input and trains the model to reconstruct it. `dropout` can be configured on branches or fields. `p_mask` and `p_prune` are configured explicitly on leaf fields. These rates do not inherit down the schema tree; broad updates are made deliberately with schema selections.

This means that the control flow is the same for pretraining and finetuning. The difference between pretraining and finetuning is configuration, not a separate model architecture.

```
import json2vec as jv


model = jv.Model(
    d_model=128,
    n_layers=4,
    n_heads=4,
    batch_size=256,
    amount=jv.Number,
    merchant=jv.Category(size=4096),
    is_fraud=jv.Category(size=2),
)
model.update(jv.where("type") == "number", p_mask=0.15)
model.update(jv.where("type") == "category", p_mask=0.05)
```

```
model = jv.Model(
    d_model=128,
    n_layers=4,
    n_heads=4,
    batch_size=256,
    amount=jv.Number,
    merchant=jv.Category(size=4096),
    is_fraud=jv.Category(target=True, size=2),
)
```

3.2. Heritage-based Forward Pass

The forward pass is easiest to understand as a flow of small packages of information through the schema tree. Internally, these packages are called `Parcels`. A parcel has an `origin`, a `destination`, and a tensor payload. Leaf fields create parcels, context encoders consume child parcels and create parent parcels, and decoders use the available parcels along a field's path to make predictions.

The pass happens in three stages.

1. **Embed every visible leaf field.** Each leaf field has a datatype-specific embedder. A categorical field, numerical field, text field, entity field, or vector field may all start with different raw tensors, but each embedder converts its field into the shared `d_model` representation. The resulting parcel is sent from the leaf field to its parent context.
2. **Encode contexts from the leaves upward.** Once a context has received parcels from its children, its encoder concatenates those child representations, runs the context-specific transformer block, and pools the result into that context's own representation. That new context parcel is then sent to its parent. This repeats from the deepest contexts up to the root, so a

clickstream event can influence a login-session embedding, the login-session embedding can influence a customer embedding, and so on.

3. **Decode trainable or pruned targets from their heritage.** A field is decoded when it has trainable targets, such as masked values during pretraining, or when the field is explicitly pruned as a supervised target. To decode that field, the model gathers the parcels produced along the field's heritage: the field itself when it is still visible, its parent context, its grandparent context, and every higher context that exists for that observation. The decoder then attends over those heritage parcels and emits datatype-specific prediction tensors.

The key idea is that a prediction is not made from the root embedding alone. It is made from the path of representations that connect the field to the root.

For example, consider:

`customer/login_sessions/clickstream_events/type`

If this field is masked, its decoder can use information from:

- `customer/login_sessions/clickstream_events/type`
- `customer/login_sessions/clickstream_events`
- `customer/login_sessions`
- `customer`

If the field is pruned instead, the leaf parcel for `customer/login_sessions/clickstream_events/type` is omitted, and the decoder must rely on the surrounding context parcels.

This gives the decoder access to local evidence and broad context at the same time. The local clickstream context may explain what happened inside the session; the login-session context may explain device and region behavior; the customer context may explain whether the behavior is unusual for that customer.

This heritage-based design is important because each target may live at a different level of the schema. A transaction-level fraud target should not be forced to decode only from a root customer vector. A customer-level target should not be forced to inspect every raw event directly. The model routes information upward through contexts, then lets each decoder attend to the representations that are relevant to its own address.

This also explains why pruning works cleanly. When a field is pruned, its own input parcel is omitted from the upward pass, preventing the model from seeing the answer. The decoder still receives the remaining heritage parcels, so it must reconstruct the hidden field from surrounding context rather than copying the original value.

4. Future Improvements

There are several important capabilities that are intentionally not yet included in `json2vec`. The current implementation is focused on proving the schema-driven modeling abstraction first. The next layer of work is about making the architecture more configurable, more efficient, and easier to operate at larger scale.

4.1. Model Architecture

4.1.1. Pretrained Encoders for Recommendation Systems

There is also a clear opportunity to use `json2vec` as a pretraining layer for recommendation systems. The goal is not necessarily to replace the recommender model itself. The goal is to create

strong user and item encoders that produce pretrained embeddings, then pass those embeddings into a recommender system in the same pipeline.

This is a natural fit because recommender data is highly structured. Users have sessions, sessions have impressions, impressions have items, items have catalog metadata, and outcomes may include clicks, purchases, ratings, dwell time, skips, saves, or churn. Items also have their own structure: text descriptions, images, prices, availability, categories, sellers, brands, reviews, and historical interaction patterns.

`json2vec` could pretrain a user encoder from raw behavioral history and an item encoder from raw catalog and interaction history. Those encoders could then emit embeddings at stable schema addresses, such as `user`, `user/sessions`, `item`, or `item/reviews`. A downstream recommender could consume those embeddings as dense inputs alongside its existing retrieval, ranking, or reranking features.

The workflow would look roughly like this:

1. Pretrain `json2vec` encoders on structured user and item observations.
2. Export or stream the resulting user and item embeddings into the recommender pipeline.
3. Train the recommender model using those embeddings as pretrained representations.
4. Optionally finetune the `json2vec` encoders and recommender model together for a task-specific objective.

This would require integration points rather than a full recommender-system rewrite:

- Stable embedding outputs for user, item, session, and catalog contexts.
- A serving path that can compute or refresh embeddings for users and items.
- Adapters that pass `json2vec` embeddings into existing retrieval or ranking models.
- Checkpoint loading that supports freezing, partial finetuning, or end-to-end finetuning.
- Benchmarks that measure whether pretrained structured embeddings improve recommender quality.

The benefit is that recommendation systems could reuse the same structured representation-learning layer as other business-data models. A team could pretrain encoders over user behavior, item metadata, inventory constraints, geography, price, time, and eligibility rules, then let the recommender model decide how to use those embeddings. This also creates a natural benchmark surface for open-source collaboration: public recommendation datasets can be expressed as `json2vec` schemas, making it easier to compare pretrained encoders without each project inventing a new feature pipeline.

4.2. Datatypes and Data Pipeline

4.2.1. Media Datatypes

`json2vec` does not currently support image, audio, or video datatypes. This is not because they are conceptually incompatible with the framework. A media datatype could follow the same plugin contract as any other datatype: load content, convert it into tensors or embeddings, decode outputs where appropriate, and contribute losses or predictions.

The difficulty is operational. Media fields require more deliberate handling of file paths, object stores, streaming reads, caching, decoding libraries, batching, variable shapes, and potentially large intermediate tensors. Images, audio clips, and videos also often rely on pretrained encoders whose compute profile is very different from a categorical or numerical field.

The likely path is to support media through datatype plugins that can wrap existing encoders. For example, an image plugin might convert a file reference into a vision-transformer embedding, while

an audio plugin might convert an object-store URI into a fixed-width acoustic representation. The core architecture should only see the resulting embedding parcel; the media plugin should own the messy loading and preprocessing details.

4.2.2. Data Source and Reader Plugins

The data pipeline also needs a broader plugin system. At the moment, support is centered around a small set of source locations and file formats. That is enough for early development, but not enough for production environments where data may come from local files, S3, databases, message queues, lakehouse tables, or internal services.

There are two related plugin boundaries to add:

- **Source plugins**, which know how to enumerate and open data from a location.
- **Reader plugins**, which know how to parse a particular format into raw `json`-like observations.

This separation matters because source and format are independent. A parquet file might live locally, in S3, or behind an internal data platform. A streaming record might arrive from a queue but still decode into the same observation shape used during batch training.

A more general data pipeline would make it easier to preserve the central promise of `json2vec`: the same schema, preprocessor, tokenizer state, and model path should be used for training, batch inference, and real-time inference.

5. Appendix

5.1. Case Study: Device Tenure

In the introduction, I mentioned a tabular feature that took eighteen months to develop. That was actually an understatement: the modeling and data engineering work took eighteen months, but only after roughly three years of prior infrastructure improvements.

The feature was "device tenure," used in an account-takeover model. At prediction time, it measured how long the customer had been associated with the device they were currently using. If the customer was transacting from a device first seen two years earlier, that was very different from a device first seen five minutes earlier.

The idea was simple, but the implementation was not. To serve the feature in real time, the system needed a large low-latency store of customer-device pairs, with the earliest observed timestamp for each pair. The scale was enormous: more than a billion unique customer-device combinations. Many of those combinations were created by VPNs and other network conditions that made the same underlying customer behavior appear as many distinct device or access patterns. Every new login or transaction could introduce a new pair, so the store had to be continuously updated while remaining available to the model-serving path.

With `json2vec`, the problem can be expressed differently. Instead of materializing one handcrafted tenure feature, the raw observation can include a history of login sessions or transactions with device identifiers and timestamps. The `entity` datatype can show the model which events used the same device, while the timestamp fields preserve when those events occurred.

That gives the model access to more raw context than a single tenure number. The hypothesis is that it may learn recurrence, frequency, and interactions with other behavior without materializing the handcrafted pair-store feature. This has not been established by the current documentation; the Device Tenure case study is explicitly an unevaluated schema sketch, not a measured replacement.